

Testing LakeTemperature

Slides by Aidan Bensch

Project Goals

- Develop a software that can systematically change the constants used in the LakeTemperature model and allow us to visualize and observe patterns in how this affects its output.
- Apply testing techniques to try and find inputs that will cause physically impossible or unrealistic.

LakeTemperature Model Interface

- Is a unit test produced by SPEL for the LakeTemperature module with a specific dataset.
- Allows us to change the values of constants normally specified in LakeCon using the file lakeparams.txt.
- Provides a file cpu_LakeTemperatureTest.txt that lists output values for comparison purposes.
- We can set values in lakeparams.txt, run elmtest.exe, and get results inside cpu_LakeTemperatureTest.txt.

Our Testing Solution

- Automates the process of inputting constants into lakeparams.txt and the process of analyzing the resulting values in cpu_LakeTemperatureTest.txt.
- Both the process of picking values for the constants and the process of analyzing the resulting outputs is extendable through sampling plugins and analysis plugins respectively.

What is a Sample?

- A sample is a set of values for the constants contained in lakeparams.txt.
- elmtest.exe is run once for every sample.

lakeparams.txt

```
betavis
0.0
za_lake
0.6
n2min
7.4999999999999993E-005
tdmax
277.0
pudz
0.0
mixfact
10.0
depthcrit
25.0
```

Example Sample

```
{
  "betavis": 0.0,
  "za_lake": 0.6,
  "n2min": 7.4999999999999993E-005,
  "tdmax": 277.0,
  "pudz": 0.0,
  "mixfact": 10.0,
  "depthcrit": 25.0
}
```

What Outputs are Received per Sample?

- Analysis plugins will receive the outputs from the `cpu_LakeTemperatureTest.txt` file in the form of a dictionary.

`cpu_LakeTemperatureTest.txt`

```
veg_ef%eflx_soil_grnd
1.0000000000000000E+036 1.0000000000000000E+036 1.0000000000000000E+036
1.0000000000000000E+036 77.59170422598278 1.0000000000000000E+036
veg_ef%eflx_gnet
1.0000000000000000E+036 1.0000000000000000E+036 1.0000000000000000E+036
1.0000000000000000E+036 76.98736546216425 1.0000000000000000E+036
```

Outputs Received by Analysis Plugins

```
{
  "veg_ef%eflx_soil_grnd": [
    1.0000000000000000E+036, 1.0000000000000000E+036, 1.0000000000000000E+036,
    1.0000000000000000E+036, 77.59170422598278, 1.0000000000000000E+036
  ],
  "veg_ef%eflx_gnet": [
    1.0000000000000000E+036, 1.0000000000000000E+036, 1.0000000000000000E+036,
    1.0000000000000000E+036, 76.98736546216425, 1.0000000000000000E+036
  ],
}
```

Sample Groups

- Represents an ordered grouping of samples
- Analysis plugins can analyze the result of multiple consecutive runs of elmtest.exe in context this way.

Sample Group Data

```
[
  {
    "betavis": 0.0,
    ...
  },
  {
    "betavis": 0.25,
    ...
  },
  {
    "betavis": 0.5,
    ...
  }
]
```

Ordered Output Data

```
[
  {
    "lakestate_vars%betaprime_col": [0.0, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.25, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.5, ...],
    ...
  }
]
```

Plugins

- Our testing framework supports two types of plugins, sampling and analysis.
- Sampling plugins produce a number of sample groups
- Analysis plugins can see the values in each sample group and the outputs from elmtest for each sample in that sample group.
- Analysis plugins can either opt to be run individually on each sample or wait to be run on the entire sample group.

Example

- Let's use one sampling plugin, "Order Sampling" and one analysis plugin "Differences".

Initial Output

```
Successfully loaded binary.

Loading sampling plugins:
  [Order Sampling]: Loaded

Loaded 1 sampling plugins successfully.

Loading analysis plugins:
  [Differences]: Loaded

Loaded 1 analysis plugins successfully.

Sampling from plugins:
  [Order Sampling]:
    (betavis_min_to_max): 11 samples
    Total: 1 groups, 11 samples

    Grand Total: 1 groups, 11 samples

Time estimated: 0:0:0:14
Would you like to continue? (Yes/No)
```